Cold Monero Signing on a Network-Less Android Device — Threat Model

> Status: **draft, seeking critique**. Nothing here is implemented. No Monero code has been written. This document exists so that people who know Monero can tell me where I am wrong before I build anything.

1. Scope and non-goals

In scope

Signing Monero transfers on an Android device that declares **zero INTERNET permission**, so that a large holding can be moved to an OTC counterparty without the spend key ever existing on a networked device.

Out of scope / non-goals

- **Not a hardware wallet.** No secure element with certified key ops. See §3 — this is a genuine downgrade and the document does not pretend otherwise.
- **Not a node.** The cold device never validates the chain. It cannot.
- **Not a general wallet.** No receiving flow, no balance view, no history. Send-only, cold, low-frequency.
- **Not a custodial service.** Nobody holds anyone else's keys at any point.

The one-sentence version

Move the *signing* step onto a device with no network stack, keep the *chain state* step on a device that does, and make the boundary between them an explicit, documented, attackable thing rather than an assumption.

2. Assets and trust boundaries

Assets

Asset	Secret?	Where it must live	Notes
Mnemonic seed	Yes	Cold device only	Source of everything below

Spend key (a)	Yes	Cold device only	Never leaves, never displayed
View key (b)	Yes	Cold device → exported once to watch-only	Export is a **deliberate, one-way** boundary crossing (§5)
Owned outputs (with amounts, via view key)	No (to view key holder)	Watch-only side	Derived by scanning
Key images	No, but linkable	Cold device computes; watch-only must **not** accumulate a full set	See attack A4
Unsigned transaction bundle	No	Crosses boundary cold → hot? No: hot → cold	Contains inputs, ring/proof data, change, fee
Signed transaction	No	Cold → watch-only → network	Public once broadcast

Trust boundaries

B1 TEE (Android Keystore) ↔ cold app process
B2 cold app process ↔ watch-only peer (QR / file, two round trips)
B3 watch-only peer ↔ Monero network / any full node
B4 cold app process ↔ human user (screen + biometric)

B2 is the boundary that does not exist in a normal wallet, and it is where this design will live or die. B1 is the boundary that makes this weaker than a hardware wallet (§3).

3. Key custody: the honest constraint

Android Keystore cannot host ed25519. It provides NIST-curve EC (P-256/P-384/P-521), RSA, AES, HMAC and (on newer devices) some ECC signing — but Monero's spend key is an ed25519 scalar, and there is no Keystore primitive that will hold or use it.

Therefore the design is **TEE-wrapped seed**, not TEE-held key:

- The mnemonic seed (or the derived spend key bytes) is encrypted with an AES-256-GCM key generated inside Android Keystore with `setUserAuthenticationRequired(true)` and non-exportable.
- The ciphertext blob lives in app-private storage. Stealing the blob alone yields nothing.
- To sign, the cold app calls Keystore to decrypt **in-process**, obtains the seed in ordinary application memory, performs software ed25519 signing, and zeroes the buffer.

What this does and does not buy:

Property	TEE-held key (Ledger, existing Engine identity keys)	TEE-wrapped seed (this design)
Key extractable by a rooted device with a memory dump	No	**Yes, during the signing window**
Key usable without user auth	No	No
Signing happens inside secure world	Yes	**No — software, in the app process**
Works for ed25519 on Android	No	Yes

Residual risks of B1, stated plainly: a compromised cold device OS can read the seed during a signing window; malware with accessibility permissions can read the screen and spoof confirmation UI. Mitigations are process-hygiene only (short window, explicit zeroisation, no background signing, biometric gate per signature). **These mitigations reduce, not eliminate, the risk.** Anyone holding more than they can afford to lose should use a hardware wallet. This document says so on purpose — the target user is someone whose alternative is "spend key lives in a hot wallet app", which is strictly worse.

> Existing, already-shipped parts of this boundary: the Vault app in this project declares zero INTERNET permission, gates every private-key operation behind BiometricPrompt, and talks to its counterpart only over a Binder channel protected by a `signature`-level permission. Identity keys there *are* Keystore-held and non-exportable (P-256 ECDSA). The Monero case is different only in §3's curve problem.

4. Attacker positions

Reusing the four positions from `SECURITY.md`, extended for this design:

#	Position	Capability
P1	Network attacker	Everything on B3; can MITM the watch-only peer's node connection
P2	Same-device attacker (cold)	Root or a malicious app on the cold device; targets B1 and B4
P3	Relay / service operator	N/A for the Monero path — there is no operator. Retained for the Engine side
P4	Malicious counterparty	The OTC buyer/seller; controls timing, can refuse to complete
P5	**Malicious watch-only peer**	**Controls the unsigned bundle entirely: inputs, ring/proof members, change address, fee, and what the cold device is shown. This is the novel position and the main subject of §6**

P5 is the one that does not exist in a single-device wallet, and the one that a hardware-wallet flow has already had to solve. §9 asks for that prior art.

5. Cold / hot responsibility split

Watch-only side (online; may be a second phone, a PC, or a hosted service the user does not trust):

- holds the **view key** (exported once from cold, over QR, by explicit user action);
- scans the chain, discovers owned outputs, tracks their spent/unspent state;
- selects inputs, chooses ring members / constructs the membership proof data, computes change address, sets fee;

- emits an **unsigned transaction bundle**;
- receives the signed bundle and broadcasts it.

Cold side (no network permission at all):

- holds seed and spend key;
- parses and **validates** the unsigned bundle (§7);
- computes key images for the inputs it is asked to spend;
- signs;
- returns the signed bundle over QR / file.

Transfer: two one-way QR passes (hot → cold: unsigned; cold → hot: signed). No bluetooth, no NFC pairing, no network. QR because it is inspectable, air-gapped, and its bandwidth limit is itself a constraint on bundle size.

6. Attack surface

Enumerated against P5 unless noted. Each row is a claim I want challenged.

#	Attack	Mechanism	Cold-side defence	Residual
A1	**Change-address substitution**	Unsigned bundle sends the bulk of the input value to an attacker address labelled “change”	Cold device must recompute the change address from its own spend key and refuse on mismatch. Change address is derived, not trusted	Low, *if* implemented
A2	**Recipient/amount substitution**	Bundle shows the user one address+amount, signs another	What-is-signed must be what-is-displayed: display values are derived from the	Low

			parsed bundle at signing time, not from a separate display field supplied by the hot side	
A3	**Fee manipulation**	Fee is implicit in Monero (inputs − outputs − change), so an inflated fee is invisible unless computed	Cold device computes fee from the parsed bundle and displays it explicitly, in XMR, with a hard cap and a "this is N% of the transfer" warning	Low
A4	**Key-image reuse / double-spend inducement**	Hot side presents an already-spent output as unspent, or presents the same output twice; cold device signs → double spend on chain, permanently linking two of the user's outputs	Cold device keeps its own **spent-set** of key images it has ever signed for, persisted locally, and refuses any input whose key image it has already emitted. This table is small (one hash per spend) and can live on the cold	Medium — the cold device only knows what *it* signed; an output spent elsewhere is undetectable. Requires the cold device to be the only signer

			device indefinitely	
A5	**Ring / proof-member poisoning**	Hot side supplies ring members that are known-spent, attacker-controlled, or of anomalous amount/timestamp, degrading unlinkability below the network norm	**Open.** Cold device cannot verify membership without chain state. Candidate mitigations in §7.4	**High — the hardest problem in this design**
A6	**Stale chain view**	Hot side supplies outdated outputs; signed tx is invalid (harmless) or spends an output already spent in a reorg	Cold device rejects on signature failure; a spent-output check is impossible without chain state	Medium — mostly a usability issue, becomes a privacy issue when combined with A4
A7	**Bundle parsing / malleability**	Two different byte encodings parse to the same logical tx; the signed one differs from the displayed one	Canonical serialisation: parse → re-serialise canonically → sign the canonical form → verify the returned bundle equals it byte-for-byte	Low if canonicalisation is strict; **high if it is not**
A8	**Confirmation-UI spoofing** (P2)	Accessibility-service malware on the cold device renders a fake confirmation screen	No mitigation available in software. Out of scope, documented	High on a compromised device

			as a hard limit	
A9	**Metadata leakage at broadcast**	The hot side learns *when* and *from where* the user signed, and which outputs were spent	Inherent. Note that the hot side already learns this from holding the view key	Accepted
A10	**View-key export compromise**	The one-time QR export of the view key is photographed / logged	User-driven, single-purpose, with an explicit warning that the view key reveals all incoming outputs and amounts to its holder, forever	Accepted with disclosure

7. The minimum the cold side must verify independently

This is the section I most want torn apart.

7.1 Must verify (non-negotiable)

- **Change address** — recompute from own spend key; reject on mismatch (A1).
- **Fee** — compute from inputs/outputs; display explicitly; enforce a configurable hard cap (A3).
- **Recipient address and amount** — parse from the bundle, display, require biometric confirmation on the exact values that will be signed (A2).
- **Key-image freshness** — refuse to sign any input whose key image is already in the local spent-set (A4).
- **Canonical form** — re-serialise and sign the canonical bytes; the returned bundle must be byte-identical after re-parse (A7).

7.2 Cannot verify without chain state

- That the inputs are in fact unspent on the current chain.
- That the ring / proof members are real, unspent, and representative.

- That the fee is at or above the current network minimum.

7.3 What I currently believe is safe to trust the hot side with

- Which outputs exist and are unspent (bounded by A4's spent-set).
- The current network fee *floor* (a wrong value here causes rejection, not loss — verify by refusing to sign below a locally configured minimum).

7.4 A5 (ring / proof poisoning) — candidate mitigations, none validated

- **(a) Statistical sanity checks**: reject members whose amounts or block heights are implausible given the real output's amount/height. Cheap, weak, but non-zero. The cold side knows the real output's amount and height from the view-key-derived data it holds.
- **(b) Deterministic member derivation**: cold device derives the member set from a seed it controls plus a compact commitment the hot side must supply (e.g. a Merkle root of eligible outputs). Moves trust from "the list" to "the commitment", which is smaller and can be cross-checked across two independent hot peers.
- **(c) Two-hot-peer cross-check**: build the bundle with two mutually distrusting watch-only peers (e.g. own laptop + a public node) and require agreement on the member set. Does not defeat a shared-ancestor attack but raises cost substantially.
- **(d) Delegate to the existing hardware-wallet protocol** if one already solves this — see §9, question 1. This is my preferred answer if it exists.

Note that the shape of A5 changes with the consensus rules in force: under ring signatures the concern is the *decoy set*; under full-chain-membership proofs the concern becomes the *proof data and the coin set commitment*. I have not verified how this interacts with the current network upgrade state, and I would rather be corrected here than guess. **This is question 2 in §9.**

8. Open design questions on the transport and UX side

- Bundle size vs. QR capacity. Under ring signatures an unsigned bundle for a multi-input transfer is large; multi-QR sequences introduce ordering and truncation attacks. What is the maximum sensible bundle, and should the cold side enforce a hard input-count limit?
- Is a *stateless* cold device acceptable, or must it persist the spent-set (A4) across reboots and app reinstalls? Persisting it means the cold device holds a growing table of key images — a metadata leak if the device is seized. Not persisting it means A4 defence is lost on reinstall. **My current lean: persist, encrypted under the same Keystore key, with the table's existence disclosed.**

- Recovery: if the cold device dies, the seed restores a normal wallet elsewhere — but the spent-set is lost, so the replacement signer cannot enforce A4. What is the right migration story?
- Should the watch-only side be a separate published app (auditable), or may it be *any* Monero wallet that can emit unsigned bundles? The latter is more useful and less controllable.

9. What I am asking the Monero community

- **Prior art.** Hardware-wallet Monero flows already sign unsigned transactions produced by an untrusted host. Where is that protocol specified, and what does it require the device to verify? If A5 is already solved there, I want to follow it rather than reinvent it. Point me at the document or the code.
- **Consensus-state accuracy.** Does the decoy-vs-membership-proof distinction in §7.4 match the network as it stands today? If I have the current state wrong, I would like to know before writing anything that depends on it.
- **Is A4's cold-side spent-set the accepted way to prevent a malicious host from inducing a double spend**, or is there something better?
- **Is the TEE-wrapped-seed model (§3) worth building at all**, given that it is strictly weaker than a hardware wallet? My argument is that the alternative in practice is a hot wallet on the same device. Is that argument sound, or is it a false comparison?
- **Anything in §6 that I have mis-analysed** — particularly A5 and A7.

10. What exists today vs. what this document describes

Item	Status
Vault app: zero INTERNET permission, TEE-held P-256 identity keys, biometric gating, signature-permission Binder channel	**Shipped** (v3.78.0)
Append-only signed ledger with full-chain verification	**Shipped**, public in core-wallet
Vulnerability disclosure policy, bounty tiers, safe harbour	**Published**, SECURITY.md
ed25519 software signing on the cold device	**Not written**
Monero transaction parsing / construction	**Not written**

Watch-only bundle protocol	**Not written — §7 and §8 are its design draft**
QR transport	**Not written**
This threat model	**Draft v0.1 — the point of publishing it is to be corrected**

Monero code written so far: **zero lines.** This table is here so that nobody has to take my word for what stage this is at.

License

This document is published under the same licence as the repository it lives in (AGPL-3.0 for Engine3.0). Comments, corrections and hostile review are all welcome; a fork with a better §7 is the best possible outcome.